# PLAYWRIGHT API
# Interview Questions & Answers

JavaScript examples • GET / POST / PUT / PATCH / DELETE
Assertions • Authentication • Chaining • Debugging • Framework design

Senior SDET / Quality Engineering Preparation

## How to use this guide

Read the interview-ready answer first, then study the code and explain why each assertion matters. Examples use Playwright Test with JavaScript ES modules. JSONPlaceholder is a public demo API; writes are simulated and are not permanently stored.

Setup: npm install -D @playwright/test. Save tests in tests/api.spec.js and run npx playwright test tests/api.spec.js.

# Contents


- 1. Fundamentals and setup
- 2. GET, POST and HTTP methods
- 3. Response validation and assertions
- 4. Authentication, headers and request data
- 5. Reuse, chaining and framework design
- 6. Negative testing, reliability and debugging
- 7. Senior-level scenario questions
- 8. Quick revision checklist


# 1. Fundamentals and setup

## 1. What is API testing in Playwright?

Answer: API testing sends HTTP requests directly to an API and validates status, headers, response body, schema and behaviour. Playwright's APIRequestContext supports API tests without opening a browser. It can also set up server-side state before UI tests.

Interview tip: API tests are usually faster and more focused than end-to-end UI tests, but they do not replace UI coverage.

## 2. What is APIRequestContext?

Answer: APIRequestContext is Playwright's API for sending HTTP requests. The built-in request fixture is provided by Playwright Test. You can also create a context with request.newContext() for custom base URLs, headers or isolated configuration.

```
import { test, expect } from '@playwright/test';

test('API context example', async ({ request }) => {
  const response = await request.get(
    'https://jsonplaceholder.typicode.com/users/1'
  );
  expect(response.ok()).toBeTruthy();
});
```

## 3. How do you set up and run an API test?

Answer: Install Playwright Test, create a test file, import test and expect, use the request fixture, then run the test using the Playwright CLI.

```
npm install -D @playwright/test
npx playwright install
npx playwright test tests/api.spec.js
```

Interview tip: API tests do not need a browser window, so headed mode is usually unnecessary.

# 2. GET, POST and HTTP methods

## 4. How do you send a GET request and validate the response?

Answer: Call request.get(url), assert the expected status, parse the JSON body, and validate important fields.

```
import { test, expect } from '@playwright/test';

test('GET user', async ({ request }) => {
  const response = await request.get(
    'https://jsonplaceholder.typicode.com/users/1'
  );

  expect(response.status()).toBe(200);
  const body = await response.json();

  expect(body.id).toBe(1);
  expect(body.name).toBe('Leanne Graham');
  expect(body.email).toContain('@');
});
```

## 5. How do you send a POST request with JSON?

Answer: Use request.post(url, { data: payload }). Playwright serializes an object as JSON. Validate the creation status and returned fields.

```
test('POST create post', async ({ request }) => {
  const payload = {
    title: 'Playwright API',
    body: 'API interview practice',
    userId: 1
  };

  const response = await request.post(
    'https://jsonplaceholder.typicode.com/posts',
    { data: payload }
  );

  expect(response.status()).toBe(201);
  const body = await response.json();

  expect(body.title).toBe(payload.title);
  expect(body.body).toBe(payload.body);
  expect(body.userId).toBe(payload.userId);
  expect(body.id).toBeTruthy();
});
```

Interview tip: JSONPlaceholder simulates creation; it does not permanently persist the new post.

## 6. How do PUT, PATCH and DELETE differ?

Answer: PUT generally replaces a resource representation and is expected to be idempotent. PATCH applies partial updates and may or may not be idempotent. DELETE is defined as idempotent in HTTP semantics, although responses can differ across repeated calls.

```
const putResponse = await request.put('/posts/1', {
  data: {
    id: 1, title: 'Updated title',
    body: 'Updated body', userId: 1
  }
});

const patchResponse = await request.patch('/posts/1', {
  data: { title: 'New title only' }
});

const deleteResponse = await request.delete('/posts/1');
```

Interview tip: Idempotency concerns intended server state; repeated calls need not return identical status codes.

## 7. How do you pass query parameters?

Answer: Use the params option instead of manually concatenating a query string.

```
const response = await request.get(
  'https://jsonplaceholder.typicode.com/posts',
  { params: { userId: 1, _limit: 5 } }
);
expect(response.status()).toBe(200);
const posts = await response.json();
expect(posts.length).toBeLessThanOrEqual(5);
```

# 3. Response validation and assertions

## 8. How do you validate status, headers and response time?

Answer: Check the expected status and important headers, then measure elapsed time against a realistic threshold. A simple timing assertion is not a substitute for load testing.

```
const start = Date.now();
const response = await request.get(
  'https://jsonplaceholder.typicode.com/users/1'
);
const elapsedMs = Date.now() - start;

expect(response.status()).toBe(200);
expect(response.headers()['content-type']).toContain('application/json');
expect(elapsedMs).toBeLessThan(3000);
```

## 9. How do you validate response fields and types?

Answer: Parse JSON and assert required fields, values, types, arrays and business rules.

```
const body = await response.json();

expect(body).toEqual(expect.objectContaining({
  id: expect.any(Number),
  name: expect.any(String),
  email: expect.stringContaining('@')
}));
```

## 10. How do you validate JSON Schema?

Answer: Use a validator such as Ajv. Compile the schema and validate the response structure. Add separate assertions for business rules.

```
import Ajv from 'ajv';

const ajv = new Ajv();
const schema = {
  type: 'object',
  required: ['id', 'name', 'email'],
  properties: {
    id: { type: 'number' },
    name: { type: 'string' },
    email: { type: 'string' }
  }
};

const validate = ajv.compile(schema);
const body = await response.json();
expect(validate(body), JSON.stringify(validate.errors)).toBe(true);
```

Interview tip: Install with npm install ajv; store real project schemas in dedicated files.

## 11. How do you test negative scenarios?

Answer: Test invalid payloads, missing required fields, malformed values, invalid IDs, unauthenticated calls and insufficient permissions. Assert the documented status and error contract.

```
const response = await request.post(
  'https://api.example.com/users',
  { data: { email: 'not-an-email' } }
);

expect(response.status()).toBe(400);
const body = await response.json();
expect(body).toHaveProperty('error');
```

Interview tip: The example endpoint is illustrative; use your application's documented error contract.

# 4. Authentication, headers and request data

## 12. How do you pass headers and bearer-token authentication?

Answer: Use the headers option. Read tokens from environment variables or a secret manager; never hard-code them or log them.

```
const token = process.env.API_TOKEN;

const response = await request.get(
  'https://api.example.com/profile',
  {
    headers: {
      Authorization: `Bearer ${token}`,
      Accept: 'application/json'
    }
  }
);
expect(response.status()).toBe(200);
```

## 13. How do you create a reusable API request context?

Answer: Create a shared helper or fixture for base URL, default headers and common request behaviour. Dispose of manually created contexts when finished.

```
import { test, expect, request } from '@playwright/test';

test('custom context', async () => {
  const api = await request.newContext({
    baseURL: 'https://jsonplaceholder.typicode.com',
    extraHTTPHeaders: { Accept: 'application/json' }
  });

  try {
    const response = await api.get('/users/1');
    expect(response.status()).toBe(200);
  } finally {
    await api.dispose();
  }
});
```

## 14. How do you pass path parameters?

Answer: Insert the resource ID into the endpoint path, and centralize endpoint construction in API client helpers.

```
const userId = 1;
const response = await request.get(
  `https://jsonplaceholder.typicode.com/users/${userId}`
);
expect(response.status()).toBe(200);
```

## 15. How do you handle cookies and session authentication?

Answer: APIRequestContext manages cookies, and requests in the same context can share cookie state. Use isolated contexts when tests need independent sessions.

Interview tip: Explain whether your framework uses bearer tokens, OAuth, cookies or another scheme.

# 5. Reuse, chaining and framework design

## 16. How do you pass an ID from one request to another?

Answer: Parse the first response, extract the ID, verify it exists, and pass it to the next request. In production, use a stateful test API for true create-then-fetch verification.

```
test('chain API requests', async ({ request }) => {
  const create = await request.post(
    'https://jsonplaceholder.typicode.com/posts',
    { data: { title: 'Test', body: 'Body', userId: 1 } }
  );

  expect(create.status()).toBe(201);
  const created = await create.json();
  expect(created.id).toBeTruthy();

  const fetch = await request.get(
    `https://jsonplaceholder.typicode.com/posts/${created.id}`
  );
  expect(fetch.status()).toBe(200);

  const fetched = await fetch.json();
  expect(fetched.id).toBe(created.id);
});
```

Interview tip: JSONPlaceholder does not persist writes, so chained retrieval may not return the newly created resource.

## 17. What should an API automation framework contain?

Answer: Separate tests, API clients, fixtures, test data, schemas, configuration, authentication utilities, assertions and reporting. Support environment configuration, safe secrets, logs, cleanup and CI.

```
tests/
  api/
    users.spec.js
    posts.spec.js
clients/
  users.api.js
  posts.api.js
fixtures/
  api.fixture.js
schemas/
  user.schema.json
data/
  test-data.js
utils/
  auth.js
playwright.config.js
```

## 18. How do you share data between tests?

Answer: Prefer independent tests and create data each test needs. Within a test, pass values via local variables or a small context object. Avoid execution-order dependencies and mutable globals.

```
const testContext = {};
testContext.userId = createdUser.id;
// Use testContext.userId in a later step of the same test.
```

## 19. How do you generate Allure reports?

Answer: Configure the Allure Playwright reporter, run tests to produce results, then generate and open the report. Redact tokens and personal data from attachments.

```
npm install -D allure-playwright allure-commandline
npx playwright test
npx allure generate allure-results --clean
npx allure open allure-report
```

Interview tip: Configure allure-playwright as a reporter in playwright.config.js.

# 6. Negative testing, reliability and debugging

## 20. How do you debug a failing API test?

Answer: Inspect URL, method, sanitized headers, payload, status, response body and correlation ID. Check environment settings, expired authentication, data dependencies and reproducibility. Never expose secrets in logs.

## 21. How do you handle an unavailable third-party API?

Answer: Mock or stub the dependency for deterministic tests, and retain integration tests against a controlled or real test environment. Validate timeout, retry, fallback and error handling. Avoid mocking every integration test.

## 22. How do you deal with retries and flaky API tests?

Answer: Find the cause first: race conditions, shared data, rate limits, eventual consistency or unstable dependencies. Use bounded retries only when justified, with backoff and logs. Do not hide product defects.

## 23. How do you run API tests in parallel safely?

Answer: Make tests independent, use unique data, avoid shared mutable state, isolate auth contexts and clean up created resources. Tune worker count to suit the test environment and API limits.

## 24. How do you validate HTTP error codes?

Answer: Assert codes documented by the contract. Common examples: 400 invalid input, 401 missing/invalid authentication, 403 insufficient permissions, 404 missing resource, 409 conflict, and 5xx server failure. Exact mappings vary.

# 7. Senior-level scenario questions

## 25. How would you test a payment API?

Answer: Cover success, invalid input, insufficient funds, expired credentials, duplicate submissions, timeouts, provider failures, refunds and reconciliation. Validate amount, currency, transaction state, idempotency and audit trail. Use sandbox systems; never trigger real payments in routine tests.

## 26. How would you test idempotency?

Answer: Repeat the operation with the same idempotency key and verify one logical result and no duplicate side effects. Also test concurrent duplicates and reusing a key with a different payload.

```
const key = `test-${Date.now()}`;
const payload = { amount: 100, currency: 'INR' };

const first = await request.post('/payments', {
  data: payload,
  headers: { 'Idempotency-Key': key }
});
const second = await request.post('/payments', {
  data: payload,
  headers: { 'Idempotency-Key': key }
});

// Assert status, transaction ID and side effects per API contract.
expect([200, 201]).toContain(first.status());
expect([200, 201]).toContain(second.status());
```

Interview tip: The endpoint is illustrative. Status alone cannot prove idempotency.

## 27. How would you test microservices?

Answer: Use unit/component tests for service logic, integration tests at service boundaries, contract tests for consumer-provider compatibility and a smaller end-to-end suite for critical business journeys. Include failure handling, observability and data consistency.

### 28. How do you integrate API tests into CI/CD?

Answer: Run fast deterministic tests on pull requests and broader suites on scheduled or release pipelines. Store secrets securely, parameterize environments, publish reports and fail the pipeline on meaningful regressions.

### 29. How do you manage test data and cleanup?

Answer: Create test data through APIs or fixtures, use unique IDs, avoid production data, and clean up test-owned resources in teardown/finally logic. Make cleanup tolerate partial setup failures.

### 30. Playwright API testing vs REST Assured?

Answer: Both send HTTP requests and validate responses. Playwright fits JavaScript/TypeScript projects and can share tooling with UI tests. REST Assured is a Java DSL suited to Java stacks. Choose based on team skills, existing framework and maintenance.

## 8. Quick revision checklist

| Area | Revise |
|---|---|
| Methods | GET, POST, PUT, PATCH, DELETE |
| Validation | Status, headers, body, schema, timing |
| Data | Query/path params, JSON body, response chaining |
| Security | Bearer tokens, cookies, env variables, redaction |
| Framework | API clients, fixtures, config, schemas, cleanup |
| Reliability | Timeouts, retries, parallelism, mocks |
| Delivery | CLI, CI/CD, Allure, logs |
| Scenarios | Payments, idempotency, microservices, outages |

## Final interview advice

For a senior role, explain not only syntax but also what you validate, why it matters, how tests behave in parallel, how secrets are protected, how failures are diagnosed and how the suite fits into CI/CD. Do not present demo-only behaviour as production behaviour.

Code examples are educational templates. Replace sample endpoints, expected statuses, payloads and cleanup calls with your application's contract. Public demo APIs may be rate-limited or return simulated data.